Lab2 实验报告

25300680221 姚卓妍

Phase-01

1. 关键汇编

```
; phase_01
mov    %rsi,%r12                # r12 = challenge
call valid_u32_fields       # 检查输入为 1 个字段
call __isoc99_sscanf        # 读 1 个无符号整数，存到 -0x14(%rbp)
mov    %r12,%rdi
call p1_compute             # eax = 期望值
cmp    %eax,-0x14(%rbp)     # 0x4020e0，与输入比较

; p1_compute：拆分 challenge 并串联三个子函数
mov    %edi,%r8d            # lo = c 的低 32 位
shr    $0x20,%rax           # hi = c >> 32
shr    $0x10,%rdx           # c >> 16
call p1_step0 / p1_step1 / p1_step2
xor    %ecx,%eax            # 返回 d ^ a

三个子函数
p1_step0: lea (%rdi,%rdi,4),%eax ; add %esi,%eax ; xor $0x7bf3,%eax
p1_step1: lea 0xc54a(%rsi),%eax   ; xor %edi,%eax
p1_step2: sub %esi,%edi ; lea 0x30ee(%rdi,%rdi,4),%eax
```

2. 恢复计算关系

口令只有 1 个整数（count = 1），全部为 uint32_t 运算，按模 2^{32} 回绕：

```
lo = c mod 2^32,  hi = c >> 32
a  = (5 • lo + hi) ^ 0x7bf3
b  = ((c >> 16) mod 2^32 + 0xc54a) ^ hi
d  = 5 • (b − lo) + 0x30ee
口令 = d ^ a
```

3. 推理过程

1. `sscanf` 之后 `cmp $0x1,%eax`，说明读入 1 个整数。
2. 输入与 `p1_compute(challenge)` 的返回值比较，所以口令就是该返回值。
3. `p1_compute` 把 `c` 拆成 `lo`、`hi`、`c >> 16`，依次传给三个子函数。
4. `lea (%rdi,%rdi,4)` 在这里是算术（乘 5），不是取地址；目的寄存器为 32 位 `eax`，高位被丢弃，所以必须用 `uint32_t` 模拟回绕。
5. `a` 暂存在 `ecx`，三个子函数都没改它，最后与 `d` 异或。
6. 验证：seed 42 下，GDB 在 `0x4020e0` 处 `p/u $eax` 与 `x/1uw $rbp-0x14` 均为 3686105567；公开测试 10/10，seed 43 通过。

4. Seed 验证

```
dellsuux2@eeswintscomp:/mnt/c/Users/DELL/Desktop/ics-2026-bomblab$ ./defuser --seed 43 --phase 1 | ./b
omb --seed 43 --phase 1
──────────────────────────────────────────────────────────────────────
BOMBLAB 2026                                    这里还没有明天

                          *
                           \
                        .-"""-.
                      .'       '.
                     /    LOOP    \
                    |               |
                     \             /
                      '-----------'

[1 当前] [2 待解除] [3 待解除] [4 待解除] [5 待解除] [6 待解除]

──────────────────────────────────────────────────────────────────────

单关调试：只记录本次通过的关卡，不播放完整结局。

第 1 关 · 维护站
终端上留着一份维护记录。
你按了一下回车，屏幕亮起，六个指示灯中只有第一个在闪。
林夏把旁边的椅子拖过来：‘我看记录，你看程序。’
终端能读到接入请求，但还没有放行。旧的口令工具已经找不到了，
留下来的程序仍然在等一个合法的输入。

Phase 1 challenge: 0x9cde98852e60034b
Input (unsigned decimal integers):
Phase 1 defused.
```

5. 运行结果

```
dellsuux2@eeswintscomp:/mnt/c/Users/DELL/Desktop/ics-2026-bomblab$ make
gcc -I. -std=c11 -O2 -Wall -Wextra -Wpedantic -Werror defuser_main.c defuser.c tutor/phase0_solution.c
 challenge.c cli.c -o defuser
</defuser --seed 42 --phase 1 | ./bomb --seed 42 --phase 1
──────────────────────────────────────────────────────────────────────
BOMBLAB 2026                                    这里还没有明天

                          *
                           \
                        .-"""-.
                      .'       '.
                     /    LOOP    \
                    |               |
                     \             /
                      '-----------'

[1 当前] [2 待解除] [3 待解除] [4 待解除] [5 待解除] [6 待解除]

──────────────────────────────────────────────────────────────────────

单关调试：只记录本次通过的关卡，不播放完整结局。

第 1 关 · 维护站
终端上留着一份维护记录。
你按了一下回车，屏幕亮起，六个指示灯中只有第一个在闪。
林夏把旁边的椅子拖过来：‘我看记录，你看程序。’
终端能读到接入请求，但还没有放行。旧的口令工具已经找不到了，
留下来的程序仍然在等一个合法的输入。

Phase 1 challenge: 0x28efe333b266f103
Input (unsigned decimal integers):
Phase 1 defused.
```

Phase-02

1. 关键汇编

```
mov       %rsi,%r13                  # r13 = challenge c
mov       $0x6,%esi
call      valid_u32_fields        # 期望 6 个字段
...
lea       -0x40(%rbp),%rdx        # sscanf 的 6 个目标地址：
lea       -0x3c(%rbp),%rcx        #      -0x40, -0x3c, -0x38, -0x34, -0x30, -0x2c
...                                  # 间隔 4 字节，即 uint32_t x[6]
call      __isoc99_sscanf
cmp       $0x6,%eax                  # 必须读到 6 个整数

# 第一个数
mov       %r13,%rax
mov       -0x40(%rbp),%esi        # esi = x[0]
shr       $0x2,%rax
and       $0x7f,%eax
cmp       %eax,%esi                  # x[0] == (c >> 2) & 0x7f

# 循环（edx = i，ecx = 移位量）
mov       $0x1c,%ecx                 # 初始移位量 28
mov       $0x1,%edx                  # i = 1
402175: mov %r13,%rax
            shr %cl,%rax             # c >> ecx
            and $0x3f,%eax           # & 63
            xor %edx,%eax            # ^ i
            lea (%rax,%rsi,4),%eax     # + 4 * 前一项
            mov -0x40(%rbp,%rdx,4),%esi     # esi = x[i]
            cmp %eax,%esi            # 0x402187：x[i] 与期望值比较
            jne 引爆
            add $1,%rdx              # i++
            add $5,%ecx              # 移位量 += 5
            cmp $6,%rdx
            jne 402175               # 未到 6 则回到循环开头
```

2. 恢复计算关系

口令是 **6 个整数**（count = 6），对应数组 x[0..5]：

x[0] = (c >> 2) & 0x7f

x[i] = (((c >> (23 + 5i)) & 0x3f) ^ i) + 4 • x[i-1],　　i = 1..5

移位量依次为 28、33、38、43、48

3. 推理过程

1. `valid_u32_fields` 的第 2 个参数是 6，`sscanf` 后 `cmp $0x6,%eax`，两处都说明口令是 6 个整数。

2. sscanf 的 6 个目标地址从 `-0x40(%rbp)` 起每隔 4 字节，所以输入在栈上是一个连续的 `uint32_t` 数组；循环里 `-0x40(%rbp,%rdx,4)` 就是按下标 `x[i]` 取元素。
3. 第一个数单独检查，由 `c >> 2` 取低 7 位得到。
4. 循环体是递推：每轮用 `ecx` 控制从 `c` 的哪一段取 6 位，与下标 `i` 异或，再加上 **4 倍的前一项**。
5. 为什么"前一项"是 `rsi`：每轮比较前 `esi` 被装入当前的 `x[i]`，比较通过后它就留给下一轮的 `lea (%rax,%rsi,4)` 使用；第一轮的 `esi` 是 `x[0]`。
6. 循环终止：`rdx` 每轮加 1，`cmp $6,%rdx` 不等于 6 则跳回，所以 i 取 1 到 5 共 5 轮；加上 `x[0]` 共 6 个数。
7. 运算宽度：`and` 之后的数不超过 63，`lea` 的结果写入 32 位 `eax`；数值很小，用 `uint32_t` 与汇编一致。

4. Seed 验证

```
dellsuux2@eeswintscomp:/mnt/c/Users/DELL/Desktop/ics-2026-bomblab$ ./defuser --seed 43 --phase 2 | ./b
omb --seed 43 --phase 2
BOMBLAB 2026                                    这里还没有明天

                        *
                         \
                      .-"""-.
                    .'       '.
                   /    LOOP    \
                  |               |
                   \             /
                    '-----------'

[1 待解除] [2 当前] [3 待解除] [4 待解除] [5 待解除] [6 待解除]

单关调试：只记录本次通过的关卡，不播放完整结局。

第 2 关 · 钟楼
钟楼没有钟声。齿轮照常转动，日期却停在昨天。
林夏翻开底座里的记录：同一天的几次校时被写在不同位置，
每一次都接着上一次的结果。你们得先确认这串记录能接起来。
窗外的送货车又绕过街角。司机还是刚才那句话：
‘今天来不及了。明天一定送到。’

Phase 2 challenge: 0x6ec624aa85685867
Input (unsigned decimal integers):
Phase 2 defused.

[1 待解除] [2 已解除] [3 待解除] [4 待解除] [5 待解除] [6 待解除]
```

5. 运行结果



Phase-03

1. 关键汇编

```
mov     %rsi,%r12              # r12 = challenge c
mov     $0x2,%esi
call    valid_u32_fields       # 期望 2 个字段
...
lea     -0x14(%rbp),%rcx       # y 的地址
lea     -0x18(%rbp),%rdx       # x 的地址
call    __isoc99_sscanf
cmp     $0x2,%eax              # 必须读到 2 个整数

mov     %r12d,%edi
and     $0x7,%edi              # edi = c & 7
cmp     %edi,-0x18(%rbp)       # x == c & 7

mov     %r12,%rax
mov     %r12,%rsi
shr     $0x3,%rax              # c >> 3
shr     $0x20,%rsi             # c >> 32
xor     %eax,%esi              # esi = (c >> 32) ^ (c >> 3)，写 32 位，高位丢弃
call    p3_dispatch            # 参数：edi = c & 7，esi = t
cmp     %eax,-0x14(%rbp)       # 0x40231e：y == 返回值
```

p3_dispatch：switch 的跳转表

```
mov     %edi,%eax              # eax = case 编号
```

```
mov     %esi,%edi             # edi = t，作为各 case 的输入
cmp     $0x7,%eax
ja      default               # 无符号大于 7 则返回 0
notrack jmp *0x405820(,%rax,8)   # 间接跳转：表[x]

case_0: lea 0x26868(%rdi,%rdi,4),%eax        # 5t + 0x26868
case_2: lea 0(,%rdi,4),%eax ; xor $0x31e4,%eax
case_3: mov %edi,%eax ; shr $1,%edi ; xor $0x7b4b,%eax ; add %edi,%eax
case_7: mov %edi,%eax ; shr $2,%edi ; xor $0xdad,%eax ; add %edi,%eax
```

2. 恢复计算关系

口令为 **2 个整数**（count = 2）：

```
x = c & 7
t = (uint32_t)((c >> 32) ^ (c >> 3))
y = f_x(t)          f_x 见上表
口令 = "x y"
```

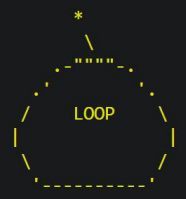
3. 推理过程

1. `valid_u32_fields` 的参数是 2，`sscanf` 后 `cmp $0x2,%eax`，所以口令是 2 个整数。
2. 第一个数 x 要等于 `c & 7`，取值 0 到 7，恰好是 `p3_dispatch` 里 `cmp $0x7` 的范围，说明它是 switch 的 case 编号。
3. `p3_dispatch` 先用 `ja`（无符号比较）把越界值送到 default，再用 `jmp *表(,%rax,8)` 间接跳转，这是 switch 的跳转表实现。
4. 表项并不直接指向 case 函数，而是指向一排 `jmp` 蹦床，且蹦床顺序被打乱，所以必须逐项读表，不能按名字猜。
5. 每个 case 的运算都由 `lea`（乘 5 加常数、乘 4）、`xor`、`shr`、`add` 组成，目的寄存器是 32 位 `eax`，所以全部按模 2^{32} 回绕，C 里用 `uint32_t`。
6. `lea 0(,%rdi,4)` 是 4t，`lea k(%rdi,%rdi,4)` 是 5t + k，`shr` 是逻辑右移。

4. seed 验证

```
dellsuux2@eeswintscomp:/mnt/c/Users/DELL/Desktop/ics-2026-bomblab$ ./defuser --seed 43 --phase 3 | ./b
omb --seed 43 --phase 3
```

BOMBLAB 2026　　这里还没有明天



[1 待解除] [2 待解除] [3 当前] [4 待解除] [5 待解除] [6 待解除]

单关调试：只记录本次通过的关卡，不播放完整结局。

第 3 关 · 分流站
沿着钟楼的线路，你们来到街口的分流站。
屏幕上的请求到达一个路口，又被送回前一个路口。
林夏在纸上划掉了一条重复路线：‘它一直在旧地址之间绕。’
站里保留着几种不同的处理办法。请求的标记决定它会走哪一条。

Phase 3 challenge: 0xd49ffe504bfdab7f
Input (unsigned decimal integers):
Phase 3 defused.

5. 运行结果

```
dellsuux2@eeswintscomp:/mnt/c/Users/DELL/Desktop/ics-2026-bomblab$ ./defuser --seed 42 --phase 3 | ./b
omb --seed 42 --phase 3
```

BOMBLAB 2026　　这里还没有明天



[1 待解除] [2 待解除] [3 当前] [4 待解除] [5 待解除] [6 待解除]

单关调试：只记录本次通过的关卡，不播放完整结局。

第 3 关 · 分流站
沿着钟楼的线路，你们来到街口的分流站。
屏幕上的请求到达一个路口，又被送回前一个路口。
林夏在纸上划掉了一条重复路线：‘它一直在旧地址之间绕。’
站里保留着几种不同的处理办法。请求的标记决定它会走哪一条。

Phase 3 challenge: 0x581ce1ff0e4ae394
Input (unsigned decimal integers):
Phase 3 defused.

Phase-04

1. 关键汇编

```
mov     $0x1,%esi
call    valid_u32_fields      # 期望 1 个字段
lea     -0x14(%rbp),%rdx
call    __isoc99_sscanf
cmp     $0x1,%eax             # 必须读到 1 个整数
```

```
mov     %r12d,%edi
and     $0x7f,%edi              # edi = a = c & 0x7f
xor     %esi,%esi               # esi = lo = 0
mov     $0x7f,%edx              # edx = hi = 127
shr     $0x7,%rcx
shr     $0x20,%rax
xor     %eax,%ecx               # ecx = k = (c>>7) ^ (c>>32)，写 32 位，高位丢弃
call    p4_walk
cmp     %eax,-0x14(%rbp)        # 0x40240a：输入 == 返回值
```

递归函数：

```
mov     %edi,%eax               # eax = a
mov     %edx,%edi
sub     %esi,%edi
shr     $1,%edi
add     %esi,%edi               # mid = lo + ((hi - lo) >> 1)
cmp     %eax,%edi
je      出口                    # mid == a
```

出口: lea 0xa4(%rdi),%eax ; xor %ecx,%eax ; ret # (mid + 0xa4) ^ k

```
lea     (%rdi,%rcx,1),%ebx      # ebx = mid + k，用被调用者保管寄存器 rbx 跨 call 保存
cmp     %edi,%eax
jb      左半                    # 无符号 a < mid
```

右半： lea 0x1(%rdi),%esi ; call p4_walk ; lea (%rax,%rax,2),%eax ; xor %ebx,%eax ; xor $0x2,%eax

左半： lea -0x1(%rdi),%edx ; call p4_walk ; lea (%rax,%rax,2),%eax ; xor %ebx,%eax ; xor $0x1,%eax

2. 恢复计算关系

Count=1

a = c & 0x7f

k = (uint32_t)((c >> 7) ^ (c >> 32))

口令 = p4_walk(a, 0, 127, k)

p4_walk(a, lo, hi, k):

mid = lo + ((hi - lo) >> 1)

若 mid == a: 返回 (mid + 0xa4) ^ k

若 a < mid: r = p4_walk(a, lo, mid-1, k); 返回 ((3r) ^ (mid + k)) ^ 1

若 a > mid: r = p4_walk(a, mid+1, hi, k); 返回 ((3r) ^ (mid + k)) ^ 2

3. 推理过程

1. `valid_u32_fields` 参数为 1，`sscanf` 后 `cmp $0x1,%eax`，口令是 1 个整数。
2. 入口给 `p4_walk` 传 `(a, 0, 127, k)`：0 和 127 是区间端点，a 取值 0 到 127，恰好落在区间内。
3. `p4_walk` 开头算中点并与 `a` 比较，相等则直接返回，这是**递归出口**；否则按 `a` 与 `mid` 的大小缩小区间，这是**二分查找**。
4. 两个递归分支的区别只有两处：区间怎么缩（`lo = mid+1` 或 `hi = mid-1`），以及返回前最后异或的常数（2 或 1）。
5. `mid + k` 在递归前算好放进 `rbx`。`rbx` 是被调用者保管寄存器，函数开头 `push`，结尾通过 `leave` 恢复，保证 `call` 回来后值没变；`rcx`（k）在整个函数里不改，所以每层 k 相同。
6. 返回值逐层合并：`lea (%rax,%rax,2)` 是 3r，再与 `mid + k` 异或，最后异或 1 或 2。这里把下层返回值层层向上加工，是递归"返回过程"的体现。
7. `a` 在区间内，每层区间缩小一半，必然在某层 `mid == a`，最多约 7 层，不会死循环.

4. Seed 验证

```
dellsuux2@eeswintscomp:/mnt/c/Users/DELL/Desktop/ics-2026-bomblab$ ./defuser --seed 43 --phase 4 | ./b
omb --seed 43 --phase 4
──────────────────────────────────────────────────────────────────────
BOMBLAB 2026                                  这里还没有明天

                          *
                           \
                        .-"""-.
                      .'       '.
                     /    LOOP    \
                    |               |
                     \             /
                      '-----------'

[1 待解除] [2 待解除] [3 待解除] [4 当前] [5 待解除] [6 待解除]

──────────────────────────────────────────────────────────────────────

单关调试：只记录本次通过的关卡，不播放完整结局。

第 4 关 · 档案馆
档案馆里没有人，书架上的指示灯一层层向深处延伸。
原来的存档经过几次搬迁，被分在不同的目录里。
你们找到入口，后面的路还要沿着旧索引逐层查下去。
林夏把刚才的纸翻到背面：‘这次别凭房间号猜，跟着记录走。’

Phase 4 challenge: 0x006ff81b72d591ff
Input (unsigned decimal integers):
Phase 4 defused.
```

5. 运行结果



Phase-05

1. 关键汇编

```
phase_05：读入 2 个数，调用 p5_traverse
mov     $0x2,%esi
call    valid_u32_fields      # 期望 2 个字段
lea     -0x18(%rbp),%rcx      # y 的地址
lea     -0x1c(%rbp),%rdx      # x 的地址
call    __isoc99_sscanf
cmp     $0x2,%eax             # 必须读到 2 个整数

lea     -0x14(%rbp),%rsi      # 第 2 个参数：输出变量 out 的地址
mov     %r12,%rdi             # 第 1 个参数：challenge c
call    p5_traverse           # 返回值 eax，同时写 out
cmp     %edx,-0x1c(%rbp)      # 0x4024dc：x == 返回值
cmp     %eax,-0x18(%rbp)      # 0x4024e4：y == out

p5_traverse：选链表头并遍历
and     $0x7,%eax                     # idx = c & 7
mov     0x405b60(,%rax,8),%rdx        # node = heads[idx]
shr     $0x6,%rax ; shr $0x20,%rdi
xor     %edi,%eax ; xor $0xf63,%eax   # ((c>>6) ^ (c>>32)) ^ 0xf63
lea     (%rax,%rax,2),%eax            # ×3
add     (%rdx),%eax                   # + node->a    （偏移 0）
mov     0x4(%rdx),%esi                # b = node->b  （偏移 4）
```

```
mov    %esi,(%r8)                  # *out = b
xor    %esi,%eax                   # acc ^= b
and    $0x3,%edi ; je 返回          # n = (c>>3)&3，为 0 直接返回
循环: mov 0x8(%rdx),%rdx           # node = node->next （偏移 8）
      lea (%rax,%rax,2),%eax       # acc = 3 • acc
      add (%rdx),%eax              # acc += node->a
      xor 0x4(%rdx),%eax           # acc ^= node->b
      cmp %ecx,%edi ; jae 循环     # 共走 n 轮
```

2. 恢复计算关系

口令为 2 个整数（count = 2），全部为 uint32_t 运算：

node = heads[c & 7]

acc = 3 • (((c>>6) ^ (c>>32)) ^ 0xf63) + node.a ; acc ^= node.b ; out = node.b

重复 n = (c>>3) & 3 次：

node = node.next

acc = 3 • acc + node.a ; acc ^= node.b ; out = node.b

口令 = "acc out"

3. 推理过程

1. valid_u32_fields 参数为 2，sscanf 后 cmp $0x2,%eax，口令是 2 个整数。
2. p5_traverse 的第 2 个参数是栈上变量的地址，函数通过 mov %esi,(%r8) 写出第二个结果，所以两个口令数一个是返回值、一个是输出参数。
3. 0x405b60(,%rax,8) 是"数组下标乘 8"，即指针数组 heads[idx]；(%rdx)、0x4(%rdx)、0x8(%rdx) 三种偏移说明节点是 a, b, next 三个字段，节点间隔 16 字节。
4. mov 0x8(%rdx),%rdx 就是 p = p->next，循环按链表前进。
5. 循环次数：ecx 初值 1，每轮先加 1 再 cmp %ecx,%edi; jae，n >= ecx 时继续，所以恰好走 n 轮（n = 0 时开头 je 直接返回）。
6. 指针是地址，C 代码中改用下标；next 为空的节点 0 在最多 3 步内不会被走到（例如 head 0 的路径是 1→2→6→5），不会空指针解引用。
7. 运算写入 32 位寄存器，高位丢弃，所以用 uint32_t 回绕。

4. Seed 验证

```
dellsuux2@eeswintscomp:/mnt/c/Users/DELL/Desktop/ics-2026-bomblab$ ./defuser --seed 42 --phase 5 | ./b
omb --seed 42 --phase 5
──────────────────────────────────────────────────────────────────────

BOMBLAB 2026                                        这里还没有明天

                              *
                               \
                            .-"""-.
                          .'       '.
                         /    LOOP    \
                        |               |
                         \             /
                          '-----------'

[1 待解除] [2 待解除] [3 待解除] [4 待解除] [5 当前] [6 待解除]

──────────────────────────────────────────────────────────────────────

单关调试：只记录本次通过的关卡，不播放完整结局。

第 5 关 · 巡检回廊
巡检回廊连着档案馆和街区。墙上的设备按编号排得很整齐，
线缆却从一台背后穿过去，接到另一排的某个接口。
维修铺的老板把一盏没装好的灯放到窗边。
底座上贴着纸条：‘今天做好接口，明天装上。’
林夏认出那盏灯：‘我们刚进来时，他就在做这个。’

Phase 5 challenge: 0xde4431fa3c80db06
Input (unsigned decimal integers):
Phase 5 defused.
```

5. 运行结果


```
dellsuux2@eeswintscomp:/mnt/c/Users/DELL/Desktop/ics-2026-bomblab$ ./defuser --seed 42 --phase 5 | ./b
omb --seed 42 --phase 5
──────────────────────────────────────────────────────────────────────

BOMBLAB 2026                                        这里还没有明天

                              *
                               \
                            .-"""-.
                          .'       '.
                         /    LOOP    \
                        |               |
                         \             /
                          '-----------'

[1 待解除] [2 待解除] [3 待解除] [4 待解除] [5 当前] [6 待解除]

──────────────────────────────────────────────────────────────────────

单关调试：只记录本次通过的关卡，不播放完整结局。

第 5 关 · 巡检回廊
巡检回廊连着档案馆和街区。墙上的设备按编号排得很整齐，
线缆却从一台背后穿过去，接到另一排的某个接口。
维修铺的老板把一盏没装好的灯放到窗边。
底座上贴着纸条：‘今天做好接口，明天装上。’
林夏认出那盏灯：‘我们刚进来时，他就在做这个。’

Phase 5 challenge: 0xde4431fa3c80db06
Input (unsigned decimal integers):
Phase 5 defused.
```


Phase-06

1. 关键汇编

phase_06：读入 1 个数，调用 p6_dispatch

```
mov     $0x1,%esi
call    valid_u32_fields      # 期望 1 个字段
lea     -0x14(%rbp),%rdx
```

```
call   __isoc99_sscanf
cmp    $0x1,%eax              # 必须读到 1 个整数
mov    %r12,%rdi              # rdi = challenge c（完整 64 位）
call   p6_dispatch
cmp    %eax,-0x14(%rbp)       # 0x402620：输入 == 返回值
p6_dispatch：两次间接调用
shr    $0x6,%rdi ; shr $0x20,%rax ; xor %eax,%edi    # t = (c>>6) ^ (c>>32)，写 32 位
and    $0x38,%ebx                                     # ebx = c & 0x38（位 3~5，已是 8 的倍数）
and    $0x7,%eax                                      # eax = c & 7
call   *0x405e00(,%rax,8)     # r = ops[c & 7](t)
xor    $0x1d9e,%eax           # r1 = r ^ 0x1d9e
mov    %eax,%edi
call   *0x405e00(%rbx)        # ops[(c>>3)&7](r1)，rbx 直接当字节偏移
add    $0xb363,%eax           # 返回值 + 0xb363
```

2. 恢复计算关系

t = (uint32_t)((c >> 6) ^ (c >> 32))

r1 = ops[c & 7](t) ^ 0x1d9e

r2 = ops[(c >> 3) & 7](r1)

口令 = r2 + 0xb363

3. 推理过程

1. `valid_u32_fields` 参数为 1，`sscanf` 后 `cmp $0x1,%eax`，口令是 1 个整数，等于 `p6_dispatch(c)`。
2. `call *0x405e00(,%rax,8)` 是"函数指针数组下标乘 8"，即 `ops[i](...)`，表项就是函数地址。
3. 第二次调用用 `*0x405e00(%rbx)`，没有乘 8，因为 `and $0x38` 取的是位 3 到 5 的值乘 8，已经是字节偏移，等价于 `ops[(c>>3)&7]`。两个下标各 3 位、互不重叠，都在 0 到 7，不会越界。
4. 第一个函数的返回值异或常数后作为第二个函数的参数，两个函数串联成流水线，数据经 eax、edi 传递。
5. `ror/rol` 是循环移位，C 里用 `(x>>k)|(x<<(32-k))` 手写；`lea` 做乘 2、乘 5 和加常数；所有结果写 32 位寄存器，用 `uint32_t` 模拟回绕。
6. 表顺序被打乱，只能用 `x/8gx` 读出地址，再与符号表地址逐项对照。

4. Seed 验证

```
dellsuux2@eeswintscomp:/mnt/c/Users/DELL/Desktop/ics-2026-bomblab$ ./defuser --seed 43 --phase 6 | ./b
omb --seed 43 --phase 6
──────────────────────────────────────────────────────────────────────
BOMBLAB 2026                                    这里还没有明天

                          *
                           \
                        .-"""-.
                      .'       '.
                     /    LOOP    \
                    |               |
                     \             /
                      '-----------'

[1 待解除] [2 待解除] [3 待解除] [4 待解除] [5 待解除] [6 当前]

──────────────────────────────────────────────────────────────────────

单关调试：只记录本次通过的关卡，不播放完整结局。

第 6 关 · 交接室
交接室的灯还亮着。桌上留着一份没有签完的记录。
你们刚才找回的时间、路线、存档和设备状态都到了这里。
林夏把记录摊开，逐项核对。你接上最后一台终端。
它会根据当前状态选择处理例程，再检查最后的交接结果。
走廊里传来送货车的声音。你们把门掩上，继续看屏幕。

Phase 6 challenge: 0x523fb4dd5d162e88
Input (unsigned decimal integers):
Phase 6 defused.

[1 待解除] [2 待解除] [3 待解除] [4 待解除] [5 待解除] [6 已解除]

dellsuux2@eeswintscomp:/mnt/c/Users/DELL/Desktop/ics-2026-bomblab$
```

5. 运行结果

```
dellsuux2@eeswintscomp:/mnt/c/Users/DELL/Desktop/ics-2026-bomblab$ ./defuser --seed 42 --phase 6 | ./b
omb --seed 42 --phase 6
──────────────────────────────────────────────────────────────────────
BOMBLAB 2026                                    这里还没有明天

                          *
                           \
                        .-"""-.
                      .'       '.
                     /    LOOP    \
                    |               |
                     \             /
                      '-----------'

[1 待解除] [2 待解除] [3 待解除] [4 待解除] [5 待解除] [6 当前]

──────────────────────────────────────────────────────────────────────

单关调试：只记录本次通过的关卡，不播放完整结局。

第 6 关 · 交接室
交接室的灯还亮着。桌上留着一份没有签完的记录。
你们刚才找回的时间、路线、存档和设备状态都到了这里。
林夏把记录摊开，逐项核对。你接上最后一台终端。
它会根据当前状态选择处理例程，再检查最后的交接结果。
走廊里传来送货车的声音。你们把门掩上，继续看屏幕。

Phase 6 challenge: 0x37e9671c45376d5d
Input (unsigned decimal integers):
Phase 6 defused.

[1 待解除] [2 待解除] [3 待解除] [4 待解除] [5 待解除] [6 已解除]
```